Updated On: May 6, 2026

STM32 FreeRTOS Event Flags Tutorial: osFlagsWaitAll, WaitAny & ISR Callbacks

Semaphores signal between tasks. **Mutexes** protect shared resources. But what do you use when you need a task to wait for *multiple* conditions at once — or react the moment *any one* of them fires?

That is the problem FreeRTOS event flags solve. An event group is a 32-bit value where each bit represents an independent event. A task can block until any one bit is set, or hold until every required bit has arrived — all through a single kernel object.

This tutorial covers everything you need to use event flags on STM32 with CMSIS-RTOS V2. We configure four tasks and an event group in STM32CubeMX on an STM32L496 Nucleo board, build two sensor-simulation examples using `osFlagsWaitAll` and `osFlagsWaitAny`, and finish with a real hardware example that sets flags directly from a GPIO button interrupt and a UART receive callback.

This is the **Part 6** of the **STM32 CMSIS-RTOS FreeRTOS series**. You can go through the other parts of this series, here are the links:

- **Part 1 —** Configure CMSIS RTOS in STM32
- **Part 2 —** Creating Multiple Tasks and Priorities in CMSIS RTOS
- **Part 3 —** How to use Queues for inter-task communication
- **Part 4 —** Using Semaphores in CMSIS RTOS

- **Part 8 —** FreeRTOS Stack Management

Table Of Contents

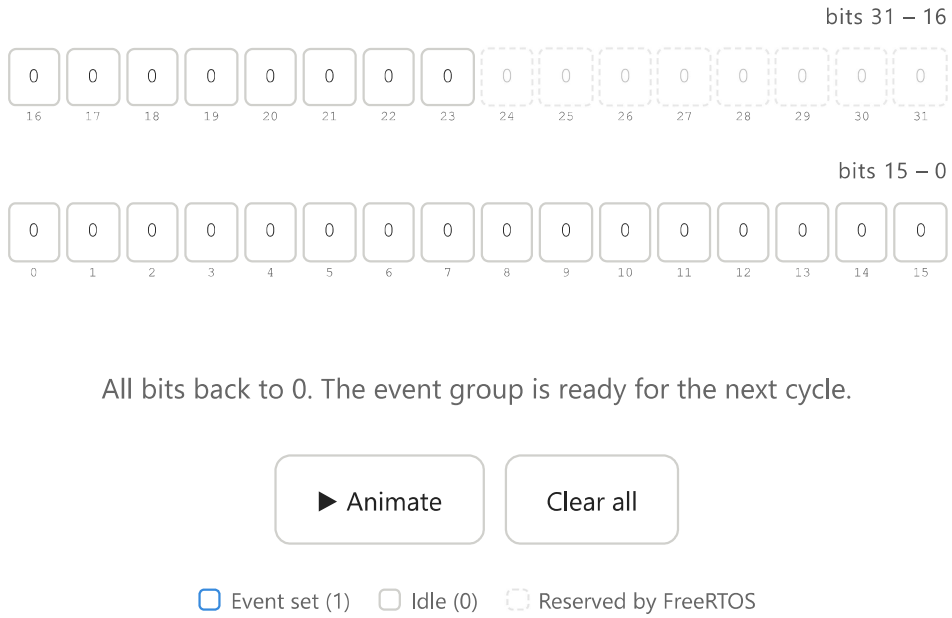
How FreeRTOS Event Flags Work on STM32

If you have worked with semaphores and mutexes, you already know how to signal between tasks and protect shared resources. But both of them

Event flags solve a different problem. They let you track multiple conditions at the same time using a single object. A task can then choose to wait for any one of those conditions, or wait until all of them are true before proceeding. This kind of flexibility is something you simply cannot achieve with a semaphore alone.

How an Event Group Works Internally

An event group is just a 32-bit integer stored in memory. Each bit in that integer represents a separate event. When an event occurs, the corresponding bit is set to 1. When the event is cleared or consumed, the bit goes back to 0.



The key thing to understand is that each bit is completely independent. Setting bit 0 does not affect bit 1 or bit 2. This means multiple events can be active at the same time, and a waiting task can choose exactly which bits it cares about.

The event group is 32 bits wide, but you do not get all 32 bits. FreeRTOS reserves the top 8 bits for its own internal use. That leaves you with **24 bits** for your application events.

In practice, 24 independent events in a single event group is more than enough for most embedded applications. And if you ever need more, you can always create a second event group.

The diagram below summarises the bit layout:

Bits	Usage
Bit 0 – Bit 23	Available for your application events
Bit 24 – Bit 31	Reserved by FreeRTOS — do not use

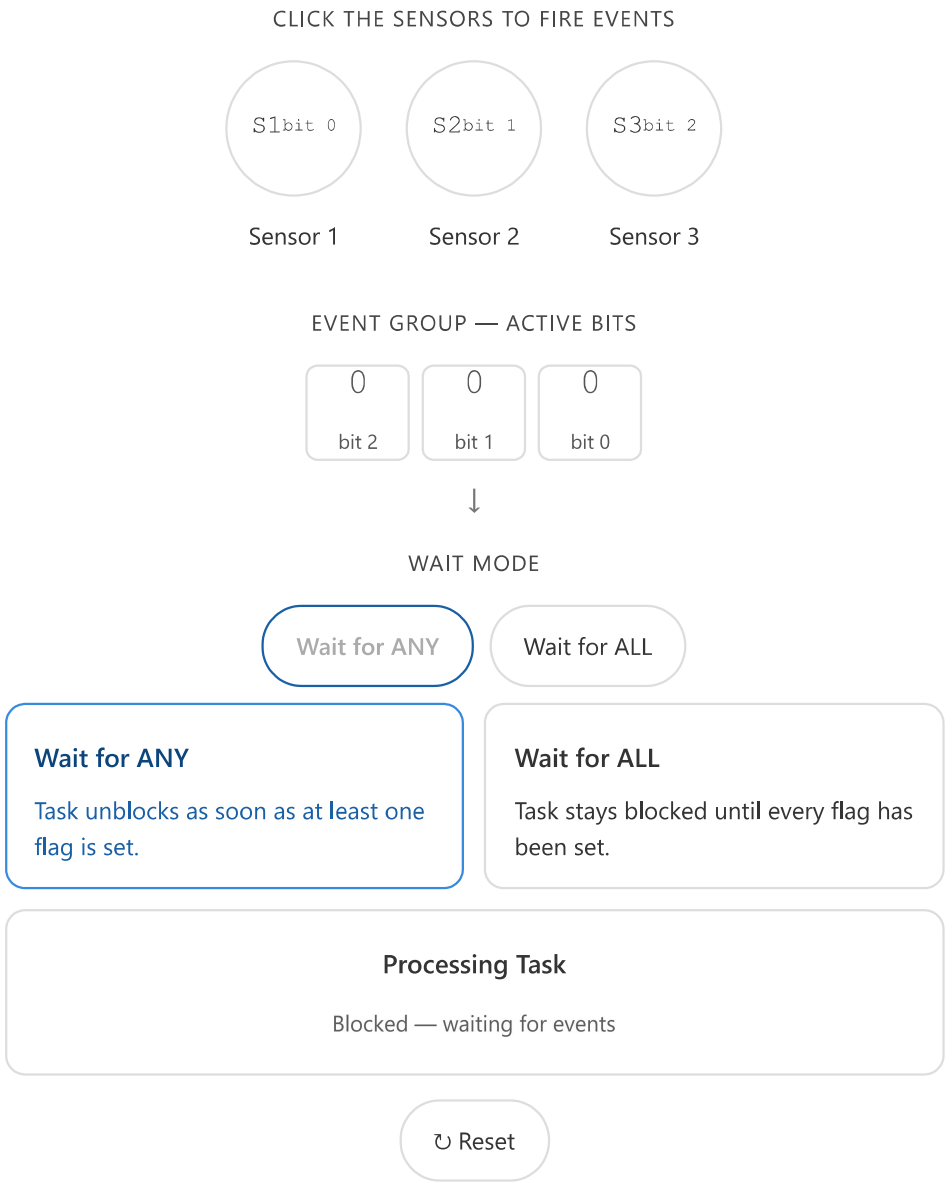
Defining Event Flags with Bit Macros

Each flag is just a unique bit position. The standard way to define them is with bit shift macros.

```
#define SENSOR1_BIT   (1 << 0)   // bit 0  →  0x00000001
#define SENSOR2_BIT   (1 << 1)   // bit 1  →  0x00000002
#define SENSOR3_BIT   (1 << 2)   // bit 2  →  0x00000004
```

When Sensor 1 fires, bit 0 gets set. When Sensor 2 fires, bit 1 gets set. Each sensor owns its own bit, and they never interfere with each other.

You can also combine flags using the OR (|) operator. For example, if you want to check whether either Sensor 1 or Sensor 2 has fired, you write



FreeRTOS Event Flags API: Set, Wait & Create

FreeRTOS provides a straightforward set of functions for working with event groups. In STM32CubeIDE with CMSIS-RTOS V2, these functions are wrapped under the `osEventFlags` prefix. We only need three of them for everything we will do in this tutorial.

Before we can use an event group, we need to create one. The function for this is:

```
1 osEventFlagsId_t osEventFlagsNew(const osEventFlagsAttr_t *
```

This allocates memory for the event group from the FreeRTOS heap and returns a handle. We use this handle in every subsequent call to set or wait for flags.

When you add an event group through the CubeMX GUI, this function is called automatically inside the main function. You will see something like this:

```
1 myEvent01Handle = osEventFlagsNew(&myEvent01_attributes);
```

You do not need to call this manually. CubeMX generates it for you.

---

osEventFlagsSet — Setting a Flag from a Task or ISR

Once the event group exists, any task or ISR can set a flag inside it using:

```
1 uint32_t osEventFlagsSet(osEventFlagsId_t ef_id, uint32_t f
```

The first parameter is the event group handle. The second is the **flag** (or combination of flags) you want to set. For example, to set only the Sensor 1 bit, we can use the respective flag:

```
1 osEventFlagsSet(myEvent01Handle, SENSOR1_BIT);
```

If you want to set multiple bits at the same time, OR (|) them together:

One important thing to note here is, this function is safe to call from both tasks and ISR callbacks. It means you can set an event flag directly inside a button interrupt or a UART receive callback without needing any special ISR-safe variant. We will use this in the practical example later.

The function returns the current value of the event group after the flags are set. You can ignore this return value in most cases.

osEventFlagsWait — Options, Parameters & Return Value

This is the most important function in the event flags API. It is where a task blocks and waits for one or more events to occur:

```
uint32_t osEventFlagsWait(osEventFlagsId_t ef_id,
                          uint32_t flags,
                          uint32_t options,
                          uint32_t timeout);
```

Let's go through each parameter carefully because the options here are what make event flags so powerful.

1. @ **`ef_id`** : The handle to our event group. This is what is returned by the function `osEventFlagsNew` .

2. @ **`flags`** : The bit(s) we want to wait for. We can specify one flag or combine multiple with OR:

```
3
4 // Wait involving multiple flags
5 osEventFlagsWait(myEvent01Handle, SENSOR1_BIT | SENSOR2_BIT
```

3. **@ `options`** : This is where we decide *how* the function waits. There are two main choices:

- `osFlagsWaitAny` — the task unblocks as soon as **at least one** of the specified flags is set. If you are waiting for three flags and only one arrives, the task wakes up immediately.
- `osFlagsWaitAll` — the task stays blocked until **every single** specified flag has been set. If you are waiting for three flags, all three must arrive before the task wakes up.

You can also add `osFlagsNoClear` to either option using OR. By default, the flags are automatically cleared after they are read. If you want to keep them set so other tasks can also read them, add this option:

```
1 osFlagsWaitAll | osFlagsNoClear
```

4. **@ `timeout`** : How long the task should wait before giving up:

- `0` — do not wait at all. If the flags are not already set, return immediately with an error.
- Any value in milliseconds — wait for that duration and return an error if the flags do not arrive in time.
- `osWaitForever` — block indefinitely until the flags are set. This is the most common choice in FreeRTOS applications.

5. **Return value** The function returns the value of the event group at the moment the task unblocked. This is very useful when you use `osFlagsWaitAny`. You can check the return value to find out which specific flag triggered the wake-up:

```
                                    osFlagsWaitAny,
                                    osWaitForever);

if (flags & SENSOR1_BIT)
{
    // Sensor 1 fired
}
else if (flags & SENSOR2_BIT)
{
    // Sensor 2 fired
}
```

If the function times out or encounters an error, the return value will have one of the error flags set (a value greater than `0x80000000`). You can check for this if your application needs error handling.

Here is a quick summary of all the options side by side:

Option	Behaviour
`osFlagsWaitAny`	Unblock when any one flag is set
`osFlagsWaitAll`	Unblock only when all flags are set
`osFlagsNoClear`	Do not clear flags after reading
`osWaitForever`	Block until flags arrive, no timeout
`0` (timeout)	Return immediately if flags not set

STM32 FreeRTOS Event Flags Example: CubeMX Setup & Code

Here we will create 4 tasks and an Event group. Three Tasks will simulate three sensors by setting respective event flags and the 4th Task read those flags.

CubeMX Setup: Tasks, Event Group & LPUART1

Creating the Tasks

Open Middleware → FreeRTOS and go to the Tasks and Queues tab. Here we will create 4 tasks. Task1, Task2, and Task3 will each simulate a sensor by setting an event flag. Task4 is the processing task — it will wait for those flags and decide what to do based on which ones are set.

The image below shows the four tasks configured in CubeMX:

Here are the details of the 4 tasks:

Task Name	Priority	Stack Size
Task1	Normal	512 words
Task2	Normal	512 words

Task3	Normal	512 words
Task4	Normal	512 words

All four tasks use the same priority and stack size because they are all doing similar lightweight work.

The stack size of 512 words is enough here because our tasks will use `printf` to log messages to the serial console. If you are not using `printf`, you can bring this down to 128 words.

Adding the Event Group

Now go to the Events tab and create a event group that all four tasks will share.

The image below shows the Event tab with one event group added.

Leave the default name of `myEvent01`. CubeMX will use this to generate the handle variable `myEvent01Handle` in the code, which we will use throughout this tutorial.

Make sure the **Allocation** is set to **Dynamic.** This means the event group will take its memory from the FreeRTOS heap at runtime rather than being allocated statically at compile time.

Configuring LPUART1 for printf Output

We will use `printf` to print the received data to a serial console. On the STM32L496 Nucleo board, the Virtual COM port routes UART data through the ST-LINK USB connection. Looking at the board schematic, ST-LINK RX is connected to **LPUART1 TX** and ST-LINK TX is connected to **LPUART1 RX.** These map to pins **PG7** and **PG8.**

Go to Connectivity and enable LPUART1 in Asynchronous mode. By default, CubeMX assigns pins PC0 and PC1, so change these to **PG7 (TX)** and **PG8 (RX)**.

Use the following settings:

Parameter	Value
Baud Rate	115200
Word Length	8 bits
Parity	None
Stop Bits	1

This is the standard UART configuration and matches what we will set in the serial console later.

Example 1: osFlagsWaitAll — Wait for All Three Sensors

In this first example, we will simulate three sensors using three separate tasks. Each task will set its own event flag and then wait. The processing task will stay blocked until all three flags are set. Only after then will it run and process the data.

Setting the Event Flags from Each Task

The first thing we need to do is define the event bits. Each task gets its own unique bit position.

```
#define TASK1_EVENT   (1 << 0)
#define TASK2_EVENT   (1 << 1)
#define TASK3_EVENT   (1 << 2)
```

Now inside each task, we print a log message to the serial console and then call `osEventFlagsSet` to set the corresponding bit. Notice that each task has a different delay — Task1 delays 1000ms, Task2 delays 2000ms, and Task3 delays 3000ms. This simulates sensors that report at different rates.

```
void StartTask1(void *argument)
{
```

```
    osEventFlagsSet(myEvent01Handle, TASK1_EVENT);
    osDelay(1000);
  }
}

void StartTask2(void *argument)
{
  for(;;)
  {
    printf("Task2 Sending Event\n");
    osEventFlagsSet(myEvent01Handle, TASK2_EVENT);
    osDelay(2000);
  }
}

void StartTask3(void *argument)
{
  for(;;)
  {
    printf("Task3 Sending Event\n");
    osEventFlagsSet(myEvent01Handle, TASK3_EVENT);
    osDelay(3000);
  }
}
```

Reading All Events in the Processing Task

Inside `StartTask4`, we call `osEventFlagsWait` with `osFlagsWaitAll`. This tells FreeRTOS to keep the task blocked until all three bits are set.

```
void StartTask4(void *argument)
{
  for(;;)
  {
    printf("Processing Task: Waiting for all sensors...\n"

    osEventFlagsWait(myEvent01Handle,
                     TASK1_EVENT | TASK2_EVENT | TASK3_EVE
                     osFlagsWaitAll,
                     osWaitForever);
```

```
14    }
15 }
```

We pass all three bits ORed together as the second parameter. The third parameter `osFlagsWaitAll` means every one of those bits must be set before the task unblocks. And `osWaitForever` as the timeout means the task will wait indefinitely.

The image below shows the serial console output after flashing the board. The processing task prints its waiting message first. Then Task1, Task2, and Task3 each send their events. Since Task3 has the longest delay of 3000ms, the processing task has to wait for it before it can proceed. Once all three events arrive, the processing task unblocks, prints the success message, and the whole cycle starts again.

This confirms that `osFlagsWaitAll` works exactly as expected. No matter how fast or slow each task fires, the processing task stays completely blocked until every single required event has arrived.

Example 2: osFlagsWaitAny — React to Any Single Event

Now let us switch to a more reactive approach. Instead of waiting for all three events, the processing task will unblock as soon as any one of the three tasks sets its flag. We also want to know exactly which task triggered it so we can handle each event independently.

We only need to make two changes from the previous example. First, we give each task a different and longer delay so they run at clearly separate intervals. Task1 runs every 3000ms, Task2 every 5000ms, and Task3 every 7000ms. This makes it easy to see on the console which task fired and when.

```
void StartTask1(void *argument)
{
  for(;;)
  {
    printf("Task1 Sending Event\n");
    osEventFlagsSet(myEvent01Handle, TASK1_EVENT);
    osDelay(3000);
  }
}

void StartTask2(void *argument)
{
  for(;;)
  {
    printf("Task2 Sending Event\n");
    osEventFlagsSet(myEvent01Handle, TASK2_EVENT);
    osDelay(5000);
  }
}

void StartTask3(void *argument)
{
  for(;;)
  {
    printf("Task3 Sending Event\n");
    osEventFlagsSet(myEvent01Handle, TASK3_EVENT);
    osDelay(7000);
  }
}
```

Checking Which Flag Was Set

`osEventFlagsWait` into a `uint32_t` variable called `flags`. This return value tells us exactly which bit was set when the task unblocked. We then check each bit individually and print the appropriate message.

```
1  void StartTask4(void *argument)
2  {
3    uint32_t flags;
4
5    for(;;)
6    {
7      printf("Processing Task: Waiting for any Event...\n");
8
9      flags = osEventFlagsWait(myEvent01Handle,
10                              TASK1_EVENT | TASK2_EVENT | T
11                              osFlagsWaitAny,
12                              osWaitForever);
13
14     if (flags & TASK1_EVENT) printf("Processing Task: Even
15     if (flags & TASK2_EVENT) printf("Processing Task: Even
16     if (flags & TASK3_EVENT) printf("Processing Task: Even
17
18     osDelay(500);
19   }
20 }
```

Output

The image below shows the serial console output. Task1 fires first at 3000ms, and the processing task immediately reacts and identifies it as Task1. Then at 5000ms, Task2 fires and the processing task reacts again. At 7000ms, Task3 fires. The processing task does not wait for the others, rather it handles each event the moment it arrives.

This is the key difference from `osFlagsWaitAll`. The processing task is now fully reactive. It wakes up for every single event independently rather than waiting for the complete set.

UART Interrupt

So far we have been generating events from tasks. But in real embedded applications, events usually come from hardware interrupts, such as a button press, a UART receive, a timer overflow. In this example we will use both.

The idea is simple. When the button is pressed, an interrupt is triggered and it sets a flag. The button task wakes up and toggles the LED. When data arrives over UART, another interrupt triggers and sets a different flag. The UART task wakes up and prints the received data. Both events are handled independently using a single event group, and the ISR callbacks stay as short as possible — they just set a flag and return immediately.

CubeMX Configuration for Button and UART Interrupt

We need to add two new tasks to the existing project in CubeMX. Go to Middleware → FreeRTOS → Tasks and Queues and add the following:

Task Name	Priority	Stack Size
ButtonTask	Normal	128 words
UartTask	Normal	512 words

ButtonTask uses a smaller stack size of 128 words because it only toggles a GPIO pin and does not use `printf`. UartTask needs 512 words because it uses `printf` to send data to the serial console.

Now configure the hardware. Go to the pin configuration and set PA3 as GPIO_EXTI3. This is the pin our button is connected to.

Set the pull to Pull-up so the pin stays high by default and goes low when the button is pressed. Then go to System Core → NVIC and **enable the interrupt** for the EXTI line 3.

For UART, the low-power UART1 is already enabled for logging. We just need to enable its global interrupt. Go to Connectivity → LPUART1 and under the NVIC Settings tab, enable the **LPUART1 global interrupt**.

Defining the Flags and the Receive Buffer

At the top of `main.c`, we define two new event bits and a buffer to hold the incoming UART data. We use bits 3 and 4 so they do not clash with the task event bits we defined earlier.

```
3
4 uint8_t RxData[10];
```

We also need to start UART reception before the FreeRTOS scheduler begins. Add this line in `main()` just before kernel initialization.

```
1 HAL_UART_Receive_IT(&hlpuart1, RxData, 5);
```

This tells the HAL library to receive 5 bytes in interrupt mode and fire the RX complete callback when done.

Setting Flags Inside the ISR Callbacks

When the button is pressed, the GPIO EXTI callback is called. We simply call `osEventFlagsSet` to set the `BUTTON_PRESSED` flag:

```
1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3   osEventFlagsSet(myEvent01Handle, BUTTON_PRESSED);
4 }
```

When 5 bytes are received over UART, the RX complete callback fires. Here we set the `UART_RECEIVED` flag and immediately re-arm the interrupt. The HAL library automatically disables the UART receive interrupt after every single callback. If we do not call `HAL_UART_Receive_IT` again here, the interrupt will only fire once and then stop working completely.

```
1 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
2 {
3   osEventFlagsSet(myEvent01Handle, UART_RECEIVED);
4   HAL_UART_Receive_IT(&hlpuart1, RxData, 5);
5 }
```

ISRs should be written in an RTOS application.

Button Task and UART Task Implementation

The button task waits for the `BUTTON_PRESSED` flag. Since we are only waiting for one flag here, it does not matter whether we use `osFlagsWaitAny` or `osFlagsWaitAll` — both behave the same way when there is only one bit involved. Once the flag arrives, we toggle pin PB7 where the LED is connected. The 500ms delay at the end also helps debounce the button.

```
1 void StartButtonTask(void *argument)
2 {
3   for(;;)
4   {
5     osEventFlagsWait(myEvent01Handle, BUTTON_PRESSED, osFla
6     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_7);
7     osDelay(500);
8   }
9 }
```

The UART task waits for the `UART_RECEIVED` flag. Once it arrives, we print the received data to the serial console.

```
1 void StartUartTask(void *argument)
2 {
3   for(;;)
4   {
5     osEventFlagsWait(myEvent01Handle, UART_RECEIVED, osFlag
6     printf("Data Received: %s\n", RxData);
7   }
8 }
```

Here is the complete code for this section:

```
/* USER CODE BEGIN PD */
#define BUTTON_PRESSED   (1 << 3)
#define UART_RECEIVED    (1 << 4)
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint8_t RxData[10];
/* USER CODE END PV */

/* Add this in main() before osKernelStart() */
HAL_UART_Receive_IT(&hlpuart1, RxData, 5);

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  osEventFlagsSet(myEvent01Handle, BUTTON_PRESSED);
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  osEventFlagsSet(myEvent01Handle, UART_RECEIVED);
  HAL_UART_Receive_IT(&hlpuart1, RxData, 5);
}

void StartButtonTask(void *argument)
{
  for(;;)
  {
    osEventFlagsWait(myEvent01Handle, BUTTON_PRESSED, osFl
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_7);
    osDelay(500);
  }
}

void StartUartTask(void *argument)
{
  for(;;)
  {
    osEventFlagsWait(myEvent01Handle, UART_RECEIVED, osFla
    printf("Data Received: %s\n", RxData);
  }
}
```

Output

The GIF below shows both events working independently. Every time the button is pressed, the LED toggles. Every time 5 bytes are sent from the computer over UART, the data appears on the serial console. Neither task interferes with the other, and the system responds immediately to each event.

This is the right way to handle hardware events in a FreeRTOS application. Keep the ISR lean, set a flag, and let the task handle the actual work. The event group ties everything together cleanly without needing a separate semaphore for each interrupt source.

Flags — Video Tutorial

Watch the complete walkthrough: event group configuration in CubeMX, defining event bits, writing task functions with osEventFlagsSet() and osEventFlagsWait(), demonstrating osFlagsWaitAll and osFlagsWaitAny, handling button and UART interrupts via ISR callbacks, and verifying all three examples on the serial console.

STM32 FreeRTOS Events — Frequently Asked Questions

+ **Can I use event flags between more than two tasks?**
+ **What happens if two tasks are both waiting on the same flag?**
+ **Does osEventFlagsSet work inside an ISR?**
+ **What happens to the flags after osEventFlagsWait returns?**

Conclusion

Event flags are the most expressive synchronisation primitive in FreeRTOS. A single event group holds up to 24 independent signal bits. Any task or ISR can set any combination of them in one atomic call. A waiting task can demand all of them at once or wake on the first one that arrives — and the return value tells it exactly which flag caused the wake-up. No polling, no semaphore chains, no global variables.

In this tutorial you built three progressively more realistic examples on the STM32 Nucleo-L496ZG-P. The `osFlagsWaitAll` example showed how a processing task can gate its execution on the arrival of all three simulated sensor events, regardless of how asynchronously they arrive. The `osFlagsWaitAny` example showed how to build a fully reactive handler that identifies and dispatches each event individually using the `osEventFlagsWait()` return value. The ISR example showed the pattern that appears in almost every real embedded project: hardware interrupts set flags, tasks respond — the ISR callbacks stay to a single line, the logic stays in task context where it has full stack space and can call `printf`, `HAL` functions, or block without consequence.

without any ISR-safe variant. This makes event flags uniquely clean for interrupt-driven architectures compared to queues (`osMessageQueuePut()` from an ISR requires careful timeout handling) and mutexes (which must never be used from an ISR at all).

The next part in this series is Part 7 — Software Timers, where you will create periodic and one-shot callbacks managed entirely by the FreeRTOS timer task — no GPIO toggle, no `osDelay()` , just a callback that fires at a precise interval. After that, Part 8 — Stack Management covers stack monitoring and overflow detection — the tools you need before deploying any FreeRTOS project to production. Browse the full **STM32 FreeRTOS series** for all parts.

View on GitHub

Support my work

Browse More STM32 FreeRTOS Tutorials

STM32 FreeRTOS Tutorial: CMSIS-RTOS V2 Setup, Tasks & LED Blink in CubeMX

STM32 FreeRTOS Multiple Tasks, Priorities & Preemption — CMSIS-RTOS V2 Guide

STM32 FreeRTOS Queue Tutorial: Inter-Task Communication with CMSIS-OS V2

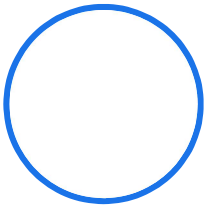
to Use Binary and Counting Semaphores

Inheritance & Recursive Mutex

Periodic & One-Shot with CMSIS-OS V2

1 2 Next »

ABOUT THE AUTHOR



Arun Rawat

EMBEDDED SYSTEMS ENGINEER · FOUNDER, CONTROLLERSTECH

Arun is an embedded systems engineer with 10+ years of experience in STM32, ESP32, and AVR microcontrollers. He created ControllersTech to share practical tutorials on embedded software, HAL drivers, RTOS, and hardware design — grounded in real industrial automation experience.

About YouTube GitHub Facebook Instagram Shop

English

{}
[+]

0 COMMENTS

Subscribe to Our Newsletter

Email

I accept the privacy policy

Subscribe